

LAPORAN MILESTONE 2

TUGAS BESAR TEORI BAHASA FORMAL DAN AUTOMATA

IF2224 – Teori Bahasa Formal dan Automata

Kelompok ZZZ – JadiApaArtiHidup?



Anggota Kelompok:

Ahmad Syafiq	13523135
Frederiko Eldad Mugiyono	13523147
Naufarrel Zhafif Abhista	13523159
Hasri Fayadh Muqaffa	13523156
I Made Wiweka Putera	13523160

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025

Daftar Isi

Daftar Isi.....	2
Landasan Teori.....	3
1. Analisis Sintaksis (Parsing).....	3
2. Context Free Grammar.....	3
3. Parse Tree.....	3
Perancangan dan Implementasi.....	4
1. Tabel Grammar.....	4
2. Struktur Proyek.....	8
3. Alur Kerja Parser.....	10
4. Penjelasan Modul Implementasi.....	10
Pengujian.....	13
1. Pengujian.....	13
Tabel 1.1. Hasil Pengujian TC1.....	14
Tabel 1.2. Hasil Pengujian TC2.....	16
Tabel 1.3. Hasil Pengujian TC3.....	19
Tabel 1.4. Hasil Pengujian TC4.....	20
Tabel 1.5. Hasil Pengujian TC5.....	24
2. Analisis Hasil Pengujian.....	24
Kesimpulan dan Saran.....	26
1. Kesimpulan.....	26
2. Saran.....	26
Lampiran.....	28
Referensi.....	30

Landasan Teori

1. Analisis Sintaksis (Parsing)

Analisis Sintaksis, atau *parsing*, adalah proses fundamental dalam kompilasi untuk menganalisis struktur gramatikal dari aliran token. Tujuan utama dari parser adalah:

1. Memeriksa apakah urutan token yang diterima dari lexer membentuk struktur sintaksis yang valid sesuai dengan aturan tata bahasa (grammar) bahasa sumber.
2. Mendeteksi dan melaporkan kesalahan sintaks (syntax error) ketika urutan token tidak sesuai dengan aturan yang diharapkan. Pesan kesalahan yang informatif sangat penting untuk membantu programmer mengidentifikasi masalah.
3. Membuat struktur data hierarkis yang merepresentasikan struktur logis dari program. Dalam konteks tugas ini, representasi tersebut adalah Parse Tree.

2. Context Free Grammar

Context-free grammar (CFG) adalah serangkaian aturan produksi yang digunakan untuk menghasilkan pola string atau kalimat dalam sebuah bahasa. CFG adalah alat formal yang digunakan dalam ilmu komputer (terutama desain kompiler dan linguistik) untuk mendeskripsikan bahasa yang lebih kompleks daripada yang dapat dijelaskan oleh tata bahasa reguler atau ekspresi reguler. Aturan produksi dalam CFG selalu berbentuk $A \rightarrow \beta$, di mana A adalah satu simbol variabel (simbol non-terminal) dan β adalah urutan dari simbol-simbol terminal, variabel, atau simbol kosong ϵ .

3. Parse Tree

Parse tree adalah struktur pohon yang merepresentasikan struktur sintaksis suatu string, seperti kalimat atau potongan kode, berdasarkan tata bahasa tertentu. Struktur ini dimulai dengan simpul akar yang merepresentasikan keseluruhan string dan memiliki simpul anak yang menguraikan struktur berdasarkan aturan tata bahasa, dengan daun pohon yang merupakan token individual atau "terminal" dari string asli.

4. Recursive Descent Parser

Recursive descent parser adalah metode *top-down* dalam *parsing* (analisis sintaksis) yang membangun pohon parsing dari atas ke bawah menggunakan serangkaian prosedur rekursif. Parser ini bekerja dengan cara menurunkan variabel-variabel dari awal hingga bertemu token terminal tanpa perlu melakukan backtrack (mengambil token kembali).

Perancangan dan Implementasi

1. Tabel Grammar

Kategori	Non-Terminal	Aturan Produksi
Program Utama	<program>	<program-header> <declaration-part> <compound-statement> DOT
	<program-header>	KEYWORD(program) IDENTIFIER SEMICOLON
Deklarasi	<declaration-part>	(<constant-declaration>)* (<type-declaration>)* (<variable-declaration>)* (<subprogram-declaration>)*
	<const-declaration>	KEYWORD(konstanta) <const-definition>+
	<const-definition>	IDENTIFIER RELATIONAL_OPERATOR(=) <expression> SEMICOLON
	<type-declaration>	KEYWORD(tipe) <type-definition>+
	<type-definition>	IDENTIFIER RELATIONAL_OPERATOR(=) <type-spec> SEMICOLON
	<var-declaration>	KEYWORD(variabel) (<var-group>)+
	<var-group>	<identifier-list> COLON <type-spec> SEMICOLON
	<identifier-list>	IDENTIFIER (COMMA IDENTIFIER)*
	<subprogram-declaration>	<procedure-declaration> <function-declaration>
	<procedure-declaration>	KEYWORD(prosedur)

		IDENTIFIER <formal-parameter-list> SEMICOLON <declaration-part> <compound-statement> SEMICOLON
	<function-declaration>	KEYWORD(fungsi) IDENTIFIER <formal-parameter-list> COLON <type-spec> SEMICOLON <declaration-part> <compound-statement> SEMICOLON
	<formal-parameter-list>	LPARENTHESIS [<formal-parameter-group> (SEMICOLON <formal-parameter-group>) *] RPARENTHESIS
	<formal-parameter-group>	<identifier-list> COLON <type-spec>
Tipe	<type-spec>	KEYWORD(integer) KEYWORD(real) KEYWORD(boolean) KEYWORD(string) KEYWORD(char) <array-type> IDENTIFIER <range>
	<array-type>	KEYWORD(larik) LBRACKET <range> RBRACKET KEYWORD(dari) <type-spec>
	<range>	<expression> RANGE_OPERATOR <expression>
Pernyataan	<statement>	<compound-statement> <if-statement> <while-statement> <for-statement> <repeat-statement> <case-statement>

		<assignment-statement> <procedure-call-statement>
	<statement-list>	[<statement> (SEMICOLON <statement>)*] [SEMICOLON]
	<compound-statement>	KEYWORD(mulai) <statement-list> KEYWORD(selesai)
	<assignment-statement>	<expression> ASSIGN_OPERATOR <expression>
	<procedure-call-statement>	<function-call-node>
	<if-statement>	KEYWORD(jika) <expression> KEYWORD(maka) <statement> [<else-clause>]
	<else-clause>	KEYWORD(selain_itu) <statement>
	<while-statement>	KEYWORD(selama) <expression> KEYWORD(lakukan) <statement>
	<for-statement>	KEYWORD(untuk) IDENTIFIER ASSIGN_OPERATOR <expression> (KEYWORD(ke) KEYWORD(turun_ke)) <expression> KEYWORD(lakukan) <statement>
	<repeat-statement>	KEYWORD(ulangi) <statement-list> KEYWORD(sampai) <expression>

	<case-statement>	KEYWORD(kasus) <expression> KEYWORD(dari) (<case-branch>)* [<case-else-clause>] KEYWORD(selesai)
	<case-branch>	<case-label-list> COLON <statement> SEMICOLON
	<case-label-list>	<expression> (COMMA <expression>)*
	<case-else-clause>	KEYWORD(selain_itu) <statement-list>
Ekspresi	<expression>	<simple-expression> (RELATIONAL_OPERATOR <simple-expression>)*
	<simple-expression>	<term> ((ARITHMETIC_OPERATOR(+ -) LOGICAL_OPERATOR(atau)) <term>)*
	<term>	<factor> ((ARITHMETIC_OPERATOR(* / bagi mod) LOGICAL_OPERATOR(dan)) <factor>)*
	<factor>	IDENTIFIER / INT_LITERAL / REAL_LITERAL / CHAR_LITERAL / STRING_LITERAL / (LPARENTHESIS + expression + RPARENTHESIS) / LOGICAL_OPERATOR(tida k) + factor / IDENTIFIER + LPARENTHESIS + <actual-parameter-list> +RPARENTHESIS / [ARITHMETIC_OPERATOR(+ -)] <factor>

	Kami melakukan modifikasi dari spesifikasi. Pada spesifikasi M2, disebutkan bahwa Unary Operator (+/-) menjadi bagian dari <code><simple-expression></code> . Namun, hal ini menyebabkan ekspresi seperti <code>a - -b</code> tidak valid (Sedangkan di pascal, ini valid). Sehingga, unary operator kami pindahkan ke <code><factor></code> .	
Expression Component	<code><function-call-node></code>	IDENTIFIER LPARENTHESIS [<code><actual-parameter-list></code>] RPARENTHESIS
	<code><array-access></code>	<code><expression></code> LBRACKET <code><expression></code> RBRACKET
	<code><parenthesized-expression></code>	LPARENTHESIS <code><expression></code> RPARENTHESIS
	<code><not-factor></code>	LOGICAL_OPERATOR(tidak) <code><factor></code>
	<code><actual-parameter-list></code>	<code><expression></code> (COMMA <code><expression></code>)*

2. Struktur Proyek

Proyek ini dan struktur folder diatur mengikuti konvensi standar dari Cargo. Adapun struktur dari proyek ini adalah sebagai berikut.

```

ZZZ-Tubes-IF2224
├── Cargo.lock
├── Cargo.toml
├── config
│   └── dfa.json
├── doc
│   ├── Diagram-1-ZZZ.png
│   ├── Laporan-1-ZZZ.pdf
│   └── Laporan-2-ZZZ.pdf
├── examples
│   ├── comment_test.pas
│   ├── comprehensive_test.pas
│   └── hello.pas
├── LICENSE
├── README.md
├── scripts
├── src
│   ├── code_generator
│   ├── interpreter
│   └── lexer

```



```

├── dfa.rs
├── lexer.rs
├── mod.rs
├── token_types.rs
├── main.rs
├── parser
│   ├── declarations.rs
│   ├── error.rs
│   ├── expressions.rs
│   ├── mod.rs
│   ├── parser.rs
│   ├── parse_tree.rs
│   ├── statements.rs
│   └── tree_printer.rs
├── semantic_analyzer
├── utils
└── test
    ├── integration
    ├── milestone-1
    │   ├── expected_output_hello.txt
    │   ├── test1_simple.pas
    │   ├── test2_operators.pas
    │   ├── test3_strings_chars.pas
    │   ├── test4_comments.pas
    │   ├── test5_arrays_range.pas
    │   ├── test_hello.pas
    │   └── test_testLexer.pas
    ├── milestone-2
    │   ├── expected_output_expression.txt
    │   ├── test1_expression.pas
    │   ├── test2_literals.pas
    │   ├── test3_all_ops.pas
    │   ├── test4_array_func.pas
    │   ├── test5_procedure.pas
    │   ├── test6_error.pas
    │   ├── test7_nested_loop.pas
    │   ├── test8_switch_case.pas
    │   └── test9_edge.pas
    ├── milestone-3
    ├── milestone-4
    └── milestone-5

```

Keterangan:

- config/: Berisi file dfa.json. File tersebut menyimpan aturan DFA yang akan dibaca oleh program rust untuk mengetahui cara mengenali token.
- doc/: Berisi dokumen spesifikasi dan laporan dari tiap milestone.

- `examples/`: Berisi contoh *source code* Pascal-S (`.pas`) yang digunakan sebagai masukan untuk compiler dan memastikan *lexical analyzer* berfungsi dengan benar.
- `src/`: Berisi *source code* utama dari tugas besar ini. Terdapat beberapa folder di dalamnya. Untuk milestone ini, fokus utamanya berada pada folder `parser`. Selain itu, terdapat file `main.rs` yang menjadi *entry point* dari program.
- `target/`: Berisi hasil kompilasi dari program yang dibuat oleh Cargo.
- `test/`: Berisi file *input* dan *output* dari file uji.
- `Cargo.toml`: Berisi manifest dari proyek ini yang mendefinisikan proyek, versi dan dependensi atau *library* eksternal yang digunakan.
- `README.md`: Berisi dokumentasi dari proyek.

3. Alur Kerja Parser

Parser bertugas memproses token yang telah diproses Lexer sehingga setiap token memiliki hubungan yang benar secara gramatikal. Dalam kode ini, alur kerja parser-nya diimplementasikan sebagai **Recursive Descent Parser**. Setiap *struct non-terminal* di `parse_tree.rs` memiliki satu (atau lebih) fungsi di `file.rs` yang sesuai untuk mem-parsing-nya. Berikut adalah alur kerja lengkap dari awal hingga akhir:

1. Inisialisasi

Alur dimulai di `main.rs`, setelah Lexer menyelesaikan tugasnya dan menghasilkan `Vec<Token>`.

1. Sebuah `struct PascalParser` dibuat, menyimpan semua token dan sebuah *pointer* (indeks *current*) untuk melacak token mana yang sedang dilihat.
2. `main.rs` memanggil `parser.parse()`.

2. Proses Recursive Descent

Panggilan `parser.parse()` ini memulai "penurunan" (descent) rekursif dari aturan gramatika tertinggi:

1. **Level Atas (<program>):**
 - a. Fungsi `parse()` memanggil `parse_program()`.
 - b. `parse_program()` bertugas mem-parsing struktur terluar: `program ... ; <declarations> <compound_statement> .`
 - c. Ia mem-parsing *header* dan menyimpannya di `node ProgramHeader`.
2. **Delegasi ke Sub-Parser:**
`parse_program()` kemudian mendelegasikan pekerjaan ke fungsi-fungsi yang lebih spesifik secara berurutan:

- a. **Declaration:** ia memanggil `parse_declaration_part()` (dari `declarations.rs`) untuk menangani semua blok konstanta, tipe, variabel, fungsi, dan prosedur.
- b. **Body:** ia memanggil `parse_compound_statement()` (dari `statements.rs`) untuk mem-parsing blok mulai ... selesai utama.
- c. **Akhir:** ia mem-validasi token DOT (titik) di akhir.

3. Router dan Hirarki:

- a. Di dalam `parse_compound_statement`, ia memanggil `parse_statement_list`, yang berulang kali memanggil `parse_statement`. Fungsi `parse_statement` bertindak sebagai **router**: ia "mengintip" (`peek()`) token berikutnya (misal jika, selama, untuk) dan memutuskan fungsi mana yang harus dipanggil (`parse_if_statement`, `parse_while_statement`, dll.).
- b. Saat parser berada di dalam sesuatu seperti `parse_if_statement`, ia perlu mem-parsing kondisi (sebuah ekspresi). Ia memanggil `parse_expression()`. Ini memicu serangkaian panggilan turun rekursif yang paling dalam untuk menangani *operator precedence*:
 - i. `parse_expression()` (untuk `<>,=`) memanggil...
 - ii. `parse_simple_expression()` (untuk `+, -, atau`) memanggil...
 - iii. `parse_term()` (untuk `*, /, dan`) memanggil...
 - iv. `parse_factor()` dan `parse_primary()` (untuk atom, *unary op*, `( )`, `[ ... ]`, dll.).

4. Rekursi:

- a. Rekursi terjadi di `expressions.rs` ketika `parse_primary` (untuk menangani `( ... )`) memanggil `parse_expression` kembali.
- b. Rekursi juga terjadi ketika `parse_declaration_part` di dalam `parse_function_declaration` memanggil `parse_declaration_part` lagi untuk menangani deklarasi *nested*.

3. Pembangunan Parse Tree

Saat parser kembali dari panggilan rekursif, ia membangun *Parse Tree/Concrete Syntax Tree* (CST).

1. Setiap fungsi parser (misal `parse_if_statement`) mengembalikan struct yang sesuai (misal `IfStatement`) dari `parse_tree.rs`.
2. struct ini diisi dengan *node-node* lain yang dikembalikan oleh fungsi yang ia panggil. `IfStatement`, misalnya, diisi dengan *node Expression* (dari `parse_expression`) dan *node Statement* (dari `parse_statement`).

3. Hasil akhir dari `parser.parse()` adalah satu *struct* `Program` yang berisi semua *node* lain yang saling *nested*, merepresentasikan seluruh hierarki *source code*.
4. Terakhir, `main.rs` mengambil *struct* `Program` (CST) ini dan menyerahkannya ke `ParseTreePrinter` untuk di-*print* ke terminal.

4. Penjelasan Modul Implementasi

Implementasi kode dibagi menjadi beberapa file dalam direktori `src/`. Pada milestone ini, fokus berada pada modul parser yang bertanggung jawab untuk melakukan analisis sintaksis. Ia mengambil daftar token dari lexer dan membangun struktur data hierarkis (dikenal sebagai Parse Tree atau Abstract Syntax Tree) berdasarkan aturan grammar Pascal-S

1) `parser/parser.rs`

File ini berisi implementasi inti dari parser. Rinciannya sebagai berikut:

- `struct PascalParser`: Menyimpan state dari parser selama proses analisis, terutama daftar token yang diterima dari lexer dan pointer ke token saat ini (`current`).
- `parse()`: Fungsi publik utama yang memulai proses parsing. Fungsi ini memanggil `parse_program()` untuk mengurai keseluruhan program.
- `parse_program()`: Mengimplementasikan aturan grammar level tertinggi (`<program>`), yang mencakup `program-header`, `declaration-part`, dan `compound-statement` (badan program).
- Fungsi Utilitas Token: Berisi berbagai fungsi helper untuk mengelola aliran token, seperti `peek()` (melihat token saat ini), `advance()` (memajukan token), `check()` (memeriksa tipe token), `consume_token()` (memajukan jika token cocok atau mengembalikan error jika tidak).

2) `parser/parse_tree.rs`

File ini mendefinisikan semua struktur data (`structs` dan `enums`) yang merepresentasikan node-node dalam Concrete Syntax Tree (CST), yang juga dikenal sebagai Parse Tree. Struktur di sini menyimpan representasi kode yang sangat rinci dan konkret, termasuk semua token sintaksis seperti kata kunci dan tanda baca.

- `struct Program`: Node root dari CST, yang secara eksplisit berisi `ProgramHeader`, `DeclarationPart`, `CompoundStatement` (badan program), dan token dot di akhir.
- `struct ProgramHeader`: Menyimpan token `program_kw`, `name` (Identifier), dan `semicolon`.
- `enum Statement`: Mendefinisikan berbagai jenis pernyataan (misalnya `Compound`, `Assignment`, `If`, `While`) dan menyimpan semua token yang

relevan secara sintaksis, seperti `if_kw`, `then_kw`, `else_kw`, `begin_kw`, `end_kw`, dll..

- Struktur lain (seperti `Expression`, `DeclarationPart`, `ArrayType`, `VariableGroup`) juga didefinisikan dengan cara yang sama rincinya, menyimpan semua token yang membentuk struktur gramatikal tersebut.

3) **parser/declarations.rs**

File ini berisi implementasi untuk mengurai bagian `<declaration-part>` dari program Pascal-S.

- `parse_declaration_part()`: Fungsi utama yang mengidentifikasi dan mem-parsing blok deklarasi (konstanta, tipe, variabel, prosedur, fungsi) secara berurutan.
- `parse_variable_declaration_block()`: Mengurai blok variabel yang berisi satu atau lebih grup variabel.
- `parse_procedure_declaration()`: Mengurai deklarasi prosedur, termasuk nama, parameter, deklarasi lokal, dan badan mulai-selesai.
- `parse_function_declaration()`: Mengurai deklarasi fungsi, termasuk nama, parameter, tipe return, deklarasi lokal, dan badan mulai-selesai.
- `parse_type_spec()`: Mengurai spesifikasi tipe data, seperti integer, real, larik [...] dari ..., atau identifier tipe kustom.

4) **parser/statements.rs**

File ini berisi implementasi untuk mengurai semua jenis pernyataan yang dapat dieksekusi, yang biasanya ditemukan di dalam blok mulai...selesai.

- `parse_statement()`: Fungsi utama yang menentukan jenis pernyataan apa yang akan diurai (misalnya jika, selama, untuk, atau assignment).
- `parse_compound_statement()`: Mengurai blok mulai ... selesai dan memanggil `parse_statement_list` untuk mengurai isinya.
- `parse_statement_list()`: Mengurai daftar pernyataan yang dipisahkan oleh titik koma (;).
- `parse_if_statement()`: Mengurai struktur jika ... maka ... selain-itu.
- `parse_while_statement()`: Mengurai struktur selama ... lakukan.
- `parse_for_statement()`: Mengurai struktur untuk ... ke/turun-ke ... lakukan.
- `parse_case_statement()`: Mengurai struktur kasus ... dari ... selain-itu ... selesai.
- `parse_assignment_or_call()`: Membedakan antara assignment (mis. `x := 10`) dan pemanggilan prosedur (mis. `writeln('halo')`).

5) **parser/expressions.rs**

File ini mengimplementasikan parser ekspresi menggunakan Recursive Descent untuk menangani urutan prioritas operator (operator precedence).

- `parse_expression()`: Aturan level terendah (prioritas terendah), menangani operator relasional (`=`, `>`, `<`).
- `parse_simple_expression()`: Menangani operator aditif (`+`, `-`, atau).
- `parse_term()`: Menangani operator multiplikatif (`*`, `/`, `bagi`, `mod`, dan).
- `parse_factor()`: Aturan level tertinggi (prioritas tertinggi), menangani literal (angka, string), IDENTIFIER, (ekspresi), operator uner (tidak), pemanggilan fungsi, dan akses array.

6) **parser/error.rs**

File ini mendefinisikan struktur data khusus untuk menangani syntax error yang terjadi selama proses parsing.

- `struct SyntaxError`: Menyimpan pesan error yang informatif, beserta lokasi (baris dan kolom) tempat error tersebut terdeteksi.
- File ini juga mengimplementasikan fungsi `new()` untuk membuat error baru dan `fmt::Display` untuk memformat pesan error secara otomatis saat dicetak.

7) **parser/output_print.rs**

File ini adalah modul utilitas yang bertanggung jawab untuk mengubah struktur Parse Tree (yang didefinisikan di `parse_tree.rs`) menjadi representasi visual pohon yang dapat dibaca manusia, seperti contoh output yang Anda berikan.

- `struct ParseTreePrinter`: Menyimpan state yang diperlukan untuk pencetakan, terutama `indent_level` dan `prefix_stack` untuk mengelola konektor pohon.
- `print_program()`: Fungsi publik utama yang secara rekursif melintasi seluruh Parse Tree (dimulai dari Program) dan membangun string keluaran.
- `get_prefix()`: Logika inti untuk menggambar garis-garis konektor pohon (`|`, `├──`, `└──`) dengan benar berdasarkan level indentasi dan posisi node (apakah node terakhir atau bukan).

Pengujian

1. Pengujian

Pada bagian ini, dilakukan serangkaian pengujian terhadap *parser* untuk memvalidasi kemampuannya dalam memverifikasi struktur gramatikal program pada bahasa Pascal. Pengujian dilakukan dengan memberikan lima berkas masukan *.pas* yang berbeda. Setiap pengujian disajikan dalam tabel di bawah, lengkap dengan *input*, *output*, dan analisisnya.

Test Case 1	
Operasi Aritmatika dan Assignment <i>test3_all_ops.pas</i>	
Input	Output

```

1  program TestAllOps;
2  variabel
3    a, b : integer;
4    flag : boolean;
5  mulai
6    a := -5;
7    b := a + 10;
8    flag := (a < b) dan benar;
9    flag := tidak (a = b)
10 selesai.

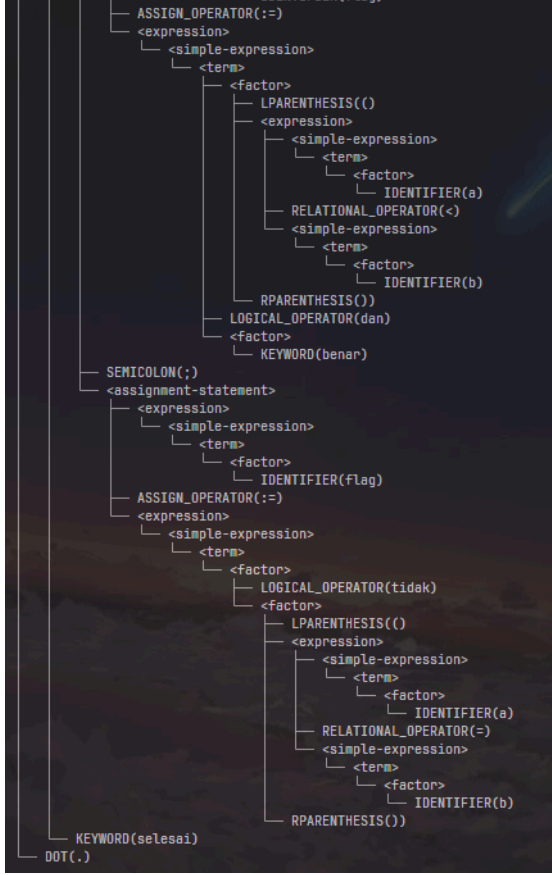
```

Parsing Berhasil!

```

<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestAllOps)
│   └── SEMICOLON(;)
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── KEYWORD(variabel)
│   │   ├── <identifier-list>
│   │   │   ├── IDENTIFIER(a)
│   │   │   ├── COMMA(,)
│   │   │   └── IDENTIFIER(b)
│   │   ├── COLON(:)
│   │   ├── <type>
│   │   │   └── KEYWORD(integer)
│   │   ├── SEMICOLON(;)
│   │   ├── <identifier-list>
│   │   │   └── IDENTIFIER(flag)
│   │   ├── COLON(:)
│   │   ├── <type>
│   │   │   └── KEYWORD(boolean)
│   │   └── SEMICOLON(;)
│   └── <compound-statement>
│       ├── KEYWORD(mulai)
│       ├── <statement-list>
│       │   ├── <assignment-statement>
│       │   │   ├── <expression>
│       │   │   │   ├── <simple-expression>
│       │   │   │   │   ├── <term>
│       │   │   │   │   │   └── <factor>
│       │   │   │   │   │       └── IDENTIFIER(a)
│       │   │   │   └── ASSIGN_OPERATOR(:=)
│       │   │   └── <expression>
│       │   │       ├── <simple-expression>
│       │   │       │   ├── ARITHMETIC_OPERATOR(-)
│       │   │       │   ├── <term>
│       │   │       │   │   ├── <factor>
│       │   │       │   │   │   └── INT_LITERAL(5)
│       │   │       └── SEMICOLON(;)
│       │   ├── <assignment-statement>
│       │   │   ├── <expression>
│       │   │   │   ├── <simple-expression>
│       │   │   │   │   ├── <term>
│       │   │   │   │   │   └── <factor>
│       │   │   │   │   │       └── IDENTIFIER(b)
│       │   │   └── ASSIGN_OPERATOR(:=)
│       │   └── <expression>
│       │       ├── <simple-expression>
│       │       │   ├── <term>
│       │       │   │   ├── <factor>
│       │       │   │   │   └── IDENTIFIER(a)
│       │       │   ├── ARITHMETIC_OPERATOR(+)
│       │       │   ├── <term>
│       │       │   │   ├── <factor>
│       │       │   │   │   └── INT_LITERAL(10)
│       │       └── SEMICOLON(;)
│       │   ├── <assignment-statement>
│       │   │   ├── <expression>
│       │   │   │   ├── <simple-expression>
│       │   │   │   │   ├── <term>
│       │   │   │   │   │   └── <factor>
│       │   │   │   │   │       └── IDENTIFIER(flag)

```


	
Analisis	
Parser berhasil membuat <i>Parse Tree</i> terhadap program dengan operator aritmatika dan <i>assignment</i> dengan benar	

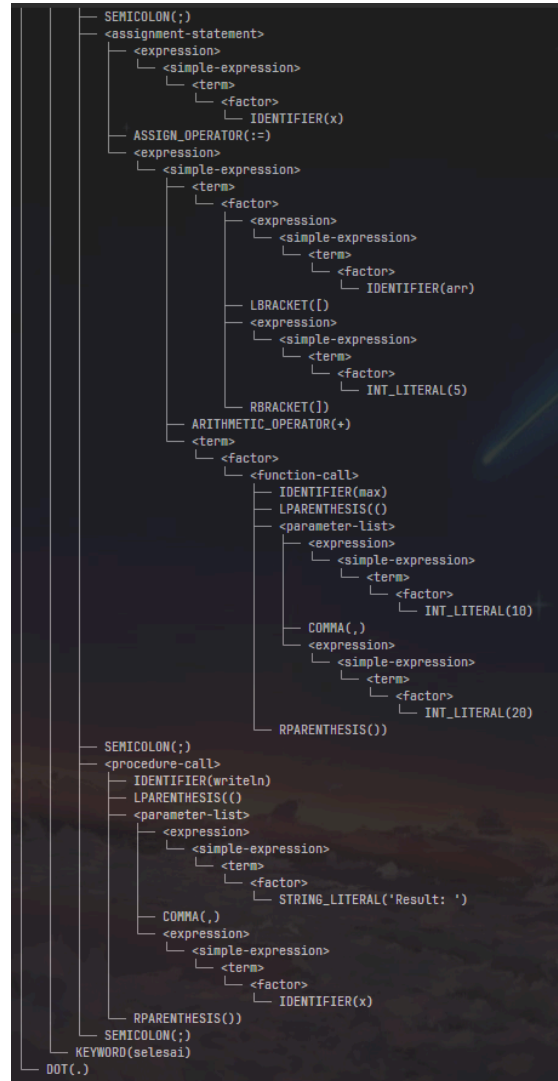
Tabel 1.1. Hasil Pengujian TC1

Test Case 2	
Array dan Function Bawaan <code>test4_array_func.pas</code>	
Input	Output



```
1 program TestArrayFunc;
2 variabel
3   arr : larik[1..10] dari intege
4   r;
5   x : integer;
6   mulai
7   arr[5] := 42;
8   x := arr[5] + max(10, 20);
9   writeln('Result: ', x);
10  selesai.
```

```
<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestArrayFunc)
│   └── SEMICOLON(;)
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── KEYWORD(variabel)
│   │   ├── <identifier-list>
│   │   │   └── IDENTIFIER(arr)
│   │   ├── COLON(:)
│   │   └── <type>
│   │       ├── <array-type>
│   │       │   ├── KEYWORD(larik)
│   │       │   ├── LBRACKET([)
│   │       │   ├── <range>
│   │       │   │   ├── <expression>
│   │       │   │   │   ├── <simple-expression>
│   │       │   │   │   │   ├── <term>
│   │       │   │   │   │   └── <factor>
│   │       │   │   │   │   └── INT_LITERAL(1)
│   │       │   │   ├── RANGE_OPERATOR(..)
│   │       │   │   └── <expression>
│   │       │   │       ├── <simple-expression>
│   │       │   │       │   ├── <term>
│   │       │   │       │   └── <factor>
│   │       │   │       └── INT_LITERAL(10)
│   │       │   └── RBRACKET(])
│   │       └── KEYWORD(dari)
│   │           └── <type>
│   │               └── KEYWORD(integer)
│   ├── SEMICOLON(;)
│   ├── <identifier-list>
│   │   └── IDENTIFIER(x)
│   ├── COLON(:)
│   └── <type>
│       └── KEYWORD(integer)
└── SEMICOLON(;)
├── <compound-statement>
│   ├── KEYWORD(mulai)
│   └── <statement-list>
│       ├── <assignment-statement>
│       │   ├── <expression>
│       │   │   ├── <simple-expression>
│       │   │   │   ├── <term>
│       │   │   │   └── <factor>
│       │   │   │       ├── <expression>
│       │   │   │       │   ├── <simple-expression>
│       │   │   │       │   │   ├── <term>
│       │   │   │       │   │   └── <factor>
│       │   │   │       │   └── IDENTIFIER(arr)
│       │   │   │       └── LBRACKET([)
│       │   │   │           ├── <expression>
│       │   │   │           │   ├── <simple-expression>
│       │   │   │           │   │   ├── <term>
│       │   │   │           │   │   └── <factor>
│       │   │   │           │   └── INT_LITERAL(5)
│       │   │   │           └── RBRACKET(])
│       │   ├── ASSIGN_OPERATOR(:=)
│       │   └── <expression>
│       │       ├── <simple-expression>
│       │       │   ├── <term>
│       │       │   └── <factor>
│       │       └── INT_LITERAL(42)
│       └── <statement-list>
```



Analisis

Parser berhasil membuat *Parse Tree* terhadap program dengan array dan fungsi bawaan dengan benar

Tabel 1.2. Hasil Pengujian TC2

Test Case 3	
Deklarasi Prosedur <code>test5_procedure.pas</code>	
Input	Output

```

1  program TestDeklarasiAman;
2
3  konstanta
4      MAKS = 100;
5      NAMA = 'Gacor King';
6
7  tipe
8      Angka = integer;
9
10 variabel
11     x : Angka;
12
13 prosedur Cetak(i: Angka);
14 variabel
15     lokal_prosedur: integer;
16 mulai
17     writeln('Angka: ', i, MAKS, NAM
18 A);
19 selesai;
20 fungsi DapatAngka(): Angka;
21 variabel
22     lokal_fungsi: integer;
23 mulai
24     DapatAngka := 99;
25 selesai;
26
27 mulai
28     x := DapatAngka();
29     Cetak(x);
30 selesai.

```

Parsing Berhasil!

```

<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(TestDeklarasiAman)
│   └── SEMICOLON(;)
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── KEYWORD(konstanta)
│   │   ├── IDENTIFIER(MAKS)
│   │   ├── RELATIONAL_OPERATOR(=)
│   │   └── <expression>
│   │       ├── <simple-expression>
│   │       │   └── <term>
│   │       │       └── <factor>
│   │       │           └── INT_LITERAL(100)
│   │   └── SEMICOLON(;)
│   ├── IDENTIFIER(NAMA)
│   ├── RELATIONAL_OPERATOR(=)
│   └── <expression>
│       ├── <simple-expression>
│       │   └── <term>
│       │       └── <factor>
│       │           └── STRING_LITERAL('Gacor King')
│   └── SEMICOLON(;)
├── <type-declaration>
│   ├── KEYWORD(tipe)
│   ├── IDENTIFIER(Angka)
│   ├── RELATIONAL_OPERATOR(=)
│   ├── <type>
│   │   └── KEYWORD(integer)
│   └── SEMICOLON(;)
├── <var-declaration>
│   ├── KEYWORD(variabel)
│   ├── <identifier-list>
│   │   └── IDENTIFIER(x)
│   ├── COLON(:)
│   ├── <type>
│   │   └── IDENTIFIER(Angka)
│   └── SEMICOLON(;)
├── <procedure-declaration>
│   ├── KEYWORD(prosedur)
│   ├── IDENTIFIER(Cetak)
│   ├── <formal-parameter-list>
│   │   ├── LPARENTHESIS(
│   │   ├── <identifier-list>
│   │   │   └── IDENTIFIER(i)
│   │   ├── COLON(:)
│   │   └── <type>
│   │       └── IDENTIFIER(Angka)
│   ├── RPARENTHESIS())
│   └── SEMICOLON(;)
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── KEYWORD(variabel)
│   │   ├── <identifier-list>
│   │   │   └── IDENTIFIER(lokal_prosedur)
│   │   ├── COLON(:)
│   │   └── <type>

```

```

└─ KEYWORD(integer)
  └─ SEMICOLON(;)
└─ <compound-statement>
  └─ KEYWORD(mulai)
  └─ <statement-list>
    └─ <procedure-call>
      └─ IDENTIFIER(writeLn)
      └─ LPARENTHESIS()
      └─ <parameter-list>
        └─ <expression>
          └─ <simple-expression>
            └─ <term>
              └─ <factor>
                └─ STRING_LITERAL('Angka: ')
        └─ COMMA(,)
        └─ <expression>
          └─ <simple-expression>
            └─ <term>
              └─ <factor>
                └─ IDENTIFIER(i)
        └─ COMMA(,)
        └─ <expression>
          └─ <simple-expression>
            └─ <term>
              └─ <factor>
                └─ IDENTIFIER(MAKS)
        └─ COMMA(,)
        └─ <expression>
          └─ <simple-expression>
            └─ <term>
              └─ <factor>
                └─ IDENTIFIER(NAMA)
      └─ RPARENTHESIS())
    └─ SEMICOLON(;)
  └─ KEYWORD(selesai)
  └─ SEMICOLON(;)
└─ <function-declaration>
  └─ KEYWORD(fungsi)
  └─ IDENTIFIER(DapatAngka)
  └─ <formal-parameter-list>
    └─ LPARENTHESIS()
    └─ RPARENTHESIS())
  └─ COLON(:)
  └─ <type>
    └─ IDENTIFIER(Angka)
  └─ SEMICOLON(;)
└─ <declaration-part>
  └─ <var-declaration>
    └─ KEYWORD(variabel)
    └─ <identifier-list>
      └─ IDENTIFIER(lokal_fungsi)
    └─ COLON(:)
    └─ <type>
      └─ KEYWORD(integer)
    └─ SEMICOLON(;)
└─ <compound-statement>
  └─ KEYWORD(mulai)
  └─ <statement-list>
    └─ <assignment-statement>
      └─ <expression>

```

Analisis	
Parser berhasil membuat <i>Parse Tree</i> terhadap program dengan deklarasi dan pemanggilan procedure dengan benar	

Tabel 1.3. Hasil Pengujian TC3

Test Case 4	
Penanganan Error Header test6_error.pas	
Input	Output

	
Analisis	
Parser mengharapkan semicolon setelah nama program, tetapi tidak ada dan langsung “mulai”, sehingga parser mengembalikan SyntaxError pada 3:1	

Tabel 1.4. Hasil Pengujian TC4

Test Case 5	
Nested Loop <code>test7_nested_loop.pas</code>	
<i>Input</i>	<i>Output</i>

```

1  program NestedControl;
2
3  variabel
4      i, j, total : integer;
5      is_valid : boolean;
6
7  mulai
8      total := 0;
9
10     untuk i := 10 turun_ke 1 lakukan
11     mulai
12         j := 1;
13         is_valid := benar;
14
15         selama (j <= i) dan is_valid lakukan
16         mulai
17             jika (i mod j) = 0 maka
18             mulai
19                 total := total + j;
20                 writeln('Faktor ditemukan: ',
21 j);
22                 selesai
23                 selain_itu
24                     is_valid := salah;
25
26                 j := j + 1;
27             selesai;
28         selesai;
29     writeln('Total akhir: ', total);
30 selesai.

```

Parsing Berhasil!

```

<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(NestedControl)
│   └── SEMICOLON(;)
└── <declaration-part>
    ├── <var-declaration>
    │   ├── KEYWORD(variabel)
    │   └── <var-group>
    │       ├── <identifier-list>
    │       │   ├── IDENTIFIER(i)
    │       │   ├── COMMA(,)
    │       │   ├── IDENTIFIER(j)
    │       │   ├── COMMA(,)
    │       │   └── IDENTIFIER(total)
    │       ├── COLON(:)
    │       ├── <type>
    │       │   ├── KEYWORD(integer)
    │       │   └── SEMICOLON(;)
    │       └── <var-group>
    │           ├── <identifier-list>
    │           │   └── IDENTIFIER(is_valid)
    │           ├── COLON(:)
    │           ├── <type>
    │           │   └── KEYWORD(boolean)
    │           └── SEMICOLON(;)
    └── <compound-statement>
        ├── KEYWORD(mulai)
        ├── <statement-list>
        │   ├── <assignment-statement>
        │   │   ├── <expression>
        │   │   │   ├── <simple-expression>
        │   │   │   │   ├── <term>
        │   │   │   │   └── <factor>
        │   │   │   │       └── IDENTIFIER(total)
        │   │   └── ASSIGN_OPERATOR(:=)
        │   ├── <expression>
        │   │   ├── <simple-expression>
        │   │   │   ├── <term>
        │   │   │   │   ├── <factor>
        │   │   │   │   └── INT_LITERAL(0)
        │   └── SEMICOLON(;)
        ├── <for-statement>
        │   ├── KEYWORD(untuk)
        │   ├── IDENTIFIER(i)
        │   ├── ASSIGN_OPERATOR(:=)
        │   ├── <expression>
        │   │   ├── <simple-expression>
        │   │   └── <term>
        └── SEMICOLON(;)

```



```

      <factor>
      <INT_LITERAL(10)>
KEYWORD(turun-ke)
<expression>
  <simple-expression>
    <term>
      <factor>
      <INT_LITERAL(1)>
KEYWORD(lakukan)
<compound-statement>
  KEYWORD(mulai)
  <statement-list>
  <assignment-statement>
    <expression>
      <simple-expression>
        <term>
          <factor>
          IDENTIFIER(j)
    ASSIGN_OPERATOR(:=)
    <expression>
      <simple-expression>
        <term>
          <factor>
          INT_LITERAL(1)
  SEMICOLON(;)
  <assignment-statement>
    <expression>
      <simple-expression>
        <term>
          <factor>
          IDENTIFIER(is_valid)
    ASSIGN_OPERATOR(:=)
    <expression>
      <simple-expression>
        <term>
          <factor>
          BOOLEAN(benar)
  SEMICOLON(;)
  <while-statement>
    KEYWORD(selama)
    <expression>
      <simple-expression>
        KEYWORD(lakukan)
    <compound-statement>
      KEYWORD(mulai)
      <statement-list>
      <if-statement>
        KEYWORD(jika)
        <expression>
          <simple-expression>
            RELATIONAL_OPERATOR(=)

```

```

      RELATIONAL_OPERATOR(<=>)
      <simple-expression>
        <term>
          <factor>
          INT_LITERAL(0)
KEYWORD(maka)
<compound-statement>
  KEYWORD(mulai)
  <statement-list>
  <assignment-statement>
    <expression>
      <simple-expression>
        <term>
          <factor>
          IDENTIFIER(total)
    ASSIGN_OPERATOR(:=)
    <expression>
      <simple-expression>
        <simple-expression>
          <term>
            <factor>
            IDENTIFIER(total)
        OPERATOR(+)
        <term>
          <factor>
          IDENTIFIER(j)
  SEMICOLON(;)
  <procedure-call>
    <function-call>
      IDENTIFIER(writeln)
      LPARENTHESIS()
      <parameter-list>
      <expression>
        <simple-expression>
          <term>
            <factor>
            STRING_LITERAL('Faktor ditemukan: ')
      COMMA(,)
      <expression>
        <simple-expression>
          <term>
            <factor>
            IDENTIFIER(j)
      RPARENTHESIS()
KEYWORD(selesai)
KEYWORD(selesai-itn)
<assignment-statement>
  <expression>
    <simple-expression>
      <term>
        <factor>

```

```

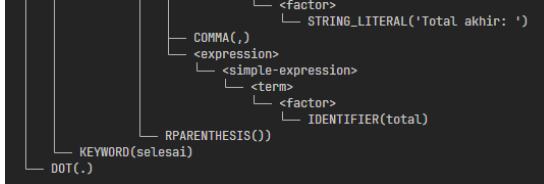
    <factor>
    IDENTIFIER(is_valid)
  ASSIGN_OPERATOR(:=)
  <expression>
  <simple-expression>
  <term>
  <factor>
  BOOLEAN(salah)
  SEMICOLON(;)
  <assignment-statement>
  <expression>
  <simple-expression>
  <term>
  <factor>
  IDENTIFIER(j)
  ASSIGN_OPERATOR(:=)
  <expression>
  <simple-expression>
  <simple-expression>
  <term>
  <factor>
  IDENTIFIER(j)
  OPERATOR(+)
  <term>
  <factor>
  INT_LITERAL(1)
  KEYWORD(selesai)
  KEYWORD(selesai)
  SEMICOLON(;)
  <procedure-call>
  <function-call>
  IDENTIFIER(writeln)
  LPARENTHESIS()
  <parameter-list>
  <expression>
  <simple-expression>
  <term>
  <factor>
  STRING_LITERAL('Total akhir: ')
  COMMA(,)
  <expression>
  <simple-expression>
  SEMICOLON(;)
  <procedure-call>
  <function-call>
  IDENTIFIER(writeln)
  LPARENTHESIS()
  <parameter-list>
  <expression>
  SEMICOLON(;)
  <procedure-call>

```

```

  <procedure-call>
  SEMICOLON(;)
  SEMICOLON(;)
  <procedure-call>
  SEMICOLON(;)
  <procedure-call>
  <function-call>
  IDENTIFIER(writeln)
  LPARENTHESIS()
  <parameter-list>
  <expression>
  <simple-expression>
  <term>
  <factor>
  STRING_LITERAL('Total akhir: ')
  COMMA(,)
  <expression>
  <simple-expression>
  <term>
  <factor>
  SEMICOLON(;)
  <procedure-call>
  <function-call>
  IDENTIFIER(writeln)
  LPARENTHESIS()
  <parameter-list>
  <expression>
  <simple-expression>
  <term>
  <factor>
  STRING_LITERAL('Total akhir: ')
  COMMA(,)
  <expression>
  <simple-expression>
  SEMICOLON(;)
  <procedure-call>
  <function-call>
  IDENTIFIER(writeln)
  LPARENTHESIS()
  <parameter-list>
  <expression>
  <simple-expression>
  SEMICOLON(;)
  <procedure-call>
  <function-call>
  IDENTIFIER(writeln)
  LPARENTHESIS()
  <parameter-list>
  <expression>
  <simple-expression>
  <term>
  <factor>
  STRING_LITERAL('Total akhir: ')

```

	
Analisis	
Parser berhasil membuat <i>Parse Tree</i> terhadap program dengan nested loop dengan benar	

Tabel 1.5. Hasil Pengujian TC5

2. Analisis Hasil Pengujian

Secara keseluruhan, hasil dari kelima kasus uji menunjukkan bahwa *parser* yang diimplementasikan telah berfungsi sesuai dengan spesifikasi yang diberikan pada *Milestone 2*. Analisis dari setiap pengujian dapat dirangkum sebagai berikut:

- a. Rangkaian pengujian yang dilakukan telah berhasil memvalidasi implementasi dari berbagai aturan grammar (CFG) Pascal-S. Ini mencakup aturan fundamental seperti deklarasi variabel (TC1), deklarasi larik (TC2), dan deklarasi prosedur (TC3).
- b. Pengujian pada nested loop (TC5) secara khusus menunjukkan keberhasilan parser recursive descent dalam menangani struktur program yang rekursif. Kemampuan parser untuk mengurai blok jika di dalam selama, yang kemudian berada di dalam untuk, membuktikan bahwa grammar untuk `<statement>` (yang dapat memanggil `<statement>` lain) telah diimplementasikan dengan tepat.
- c. Kasus uji TC1, TC2, dan TC5 memvalidasi kemampuan parser untuk mengurai `<expression>` dengan benar. Parser berhasil membedakan antara assignment (TC1) dan pemanggilan fungsi (TC2), serta mengurai ekspresi boolean majemuk di dalam kondisi loop (TC5).
- d. Kasus uji TC4 secara eksplisit menunjukkan bahwa parser dapat menangani error dengan baik, sesuai spesifikasi. Parser berhasil mendeteksi token yang tidak terduga (KEYWORD(mulai)) dan melaporkan dengan jelas token apa yang sebenarnya diharapkan (SEMICOLON(;)) dengan lokasinya (3, 1).

Berdasarkan analisis tersebut, dapat disimpulkan bahwa implementasi analisis sintaksis telah memenuhi target utama pada *Milestone 2*. Parser mampu memvalidasi struktur program Pascal-S dan menghasilkan *Parse Tree* yang akurat, yang siap digunakan untuk tahap analisis semantik.

Kesimpulan dan Saran

1. Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa tahap analisis sintaksis (parser) untuk compiler Pascal-S telah berhasil diselesaikan sesuai dengan target pada Milestone 2.

Parser yang dikembangkan dengan metode Recursive Descent terbukti mampu mengonsumsi aliran token yang dihasilkan oleh lexer dan memvalidasi struktur gramatikal program.

Implementasi ini telah berhasil:

1. Parser secara akurat mengimplementasikan aturan tata bahasa bebas konteks untuk Pascal-S, termasuk penanganan kata kunci Bahasa Indonesia.
2. Parser mampu mengenali dan membangun struktur untuk semua konstruk bahasa yang disyaratkan, mulai dari <program-header>, bagian <declaration-part> (termasuk variabel, prosedur, fungsi, konstanta, dan tipe), hingga berbagai <statement> kompleks (seperti jika-selain-itu, selama, untuk, kasus, dan ulangi).
3. Pengujian (seperti TC5) menunjukkan kemampuan parser dalam menangani struktur bersarang (nested) secara rekursif, misalnya blok jika di dalam loop selama.
4. Keluaran dari parser adalah Parse Tree yang akurat dan hierarkis, yang merepresentasikan struktur sintaksis program dan siap untuk diproses oleh tahap selanjutnya.
5. Parser mampu mendeteksi kesalahan sintaksis, seperti token yang hilang atau tidak terduga (TC4), dan melaporkan error dengan pesan dan lokasi yang informatif.

2. Saran

Meskipun fungsionalitas utama untuk Milestone 2 telah tercapai, ada beberapa saran dan langkah pengembangan logis untuk tahap berikutnya (Milestone 3: Analisis Semantik):

1. Tahap selanjutnya adalah membangun analyzer semantik yang akan mengonsumsi Parse Tree yang dihasilkan oleh parser ini.
2. Langkah krusial pertama untuk analisis semantik adalah mengimplementasikan Symbol Table (Tabel Simbol). Struktur data ini akan sangat penting untuk melacak identifer (variabel, fungsi, dll.), cakupan (scope), dan tipenya.
3. Saat ini parser berhenti pada syntax error pertama. Untuk compiler yang lebih robust, dapat dipertimbangkan untuk mengimplementasikan mode "panic"

dan "synchronization" agar parser dapat melanjutkan parsing setelah menemukan kesalahan untuk mendeteksi lebih dari satu error dalam sekali jalan.

Lampiran

- Link Release Repository:
<https://github.com/reletz/ZZZ-Tubes-IF2224/releases/tag/v0.2.1>
- Link Repository: <https://github.com/reletz/ZZZ-Tubes-IF2224>
- Tabel Pembagian Tugas sebagai berikut:

No	Nim	Nama	Tugas	Persentase
1	13523135	Ahmad Syafiq	Declarations	20%
2	13523147	Frederiko Eldad Mugiyono	Statements	15%
3	13523149	Naufarrel Zhafif Abhista	- Parser - Parse Tree (Node)	25%
4	13523156	Hasri Fayadh Muqaffa	Tree Printer	15%
5	13523160	IMade Wiweka Putera	- Expressions - Statements	25%

- Grammar:

Kategori	Non-Terminal	Aturan Produksi
Program Utama	<program>	<program-header> <declaration-part> <compound-statement> DOT
	<program-header>	KEYWORD(program) IDENTIFIER SEMICOLON
Deklarasi	<declaration-part>	(<constant-declaration>)* (<type-declaration>)* (<variable-declaration>)* (<subprogram-declaration>)*
	<const-declaration>	KEYWORD(konstanta)

		<const-definition>+
	<const-definition>	IDENTIFIER RELATIONAL_OPERATOR(=) <expression> SEMICOLON
	<type-declaration>	KEYWORD(type) <type-definition>+
	<type-definition>	IDENTIFIER RELATIONAL_OPERATOR(=) <type-spec> SEMICOLON
	<var-declaration>	KEYWORD(variabel) (<var-group>)+
	<var-group>	<identifier-list> COLON <type-spec> SEMICOLON
	<identifier-list>	IDENTIFIER (COMMA IDENTIFIER)*
	<subprogram-declaration>	<procedure-declaration> <function-declaration>
	<procedure-declaration>	KEYWORD(prosedur) IDENTIFIER <formal-parameter-list> SEMICOLON <declaration-part> <compound-statement> SEMICOLON
	<function-declaration>	KEYWORD(fungsi) IDENTIFIER <formal-parameter-list> COLON <type-spec> SEMICOLON <declaration-part> <compound-statement> SEMICOLON
	<formal-parameter-list>	LPARENTHESIS [<formal-parameter-group> (SEMICOLON <formal-parameter-group>)*] RPARENTHESIS

	<formal-parameter-group>	<identifier-list> COLON <type-spec>
Tipe	<type-spec>	KEYWORD(integer) KEYWORD(real) KEYWORD(boolean) KEYWORD(string) KEYWORD(char) <array-type> IDENTIFIER <range>
	<array-type>	KEYWORD(larik) LBRACKET <range> RBRACKET KEYWORD(dari) <type-spec>
	<range>	<expression> RANGE_OPERATOR <expression>
Pernyataan	<statement>	<compound-statement> <if-statement> <while-statement> <for-statement> <repeat-statement> <case-statement> <assignment-statement> <procedure-call-statement> >
	<statement-list>	[<statement> (SEMICOLON <statement>)*] [SEMICOLON]
	<compound-statement>	KEYWORD(mulai) <statement-list> KEYWORD(selesai)
	<assignment-statement>	<expression> ASSIGN_OPERATOR <expression>
	<procedure-call-statement>	<function-call-node>
	<if-statement>	KEYWORD(jika) <expression> KEYWORD(maka) <statement>

		[<else-clause>]
	<else-clause>	KEYWORD(selain_itu) <statement>
	<while-statement>	KEYWORD(selama) <expression> KEYWORD(lakukan) <statement>
	<for-statement>	KEYWORD(untuk) IDENTIFIER ASSIGN_OPERATOR <expression> (KEYWORD(ke) KEYWORD(turun_ke)) <expression> KEYWORD(lakukan) <statement>
	<repeat-statement>	KEYWORD(ulangi) <statement-list> KEYWORD(sampai) <expression>
	<case-statement>	KEYWORD(kasus) <expression> KEYWORD(dari) (<case-branch>)* [<case-else-clause>] KEYWORD(selesai)
	<case-branch>	<case-label-list> COLON <statement> SEMICOLON
	<case-label-list>	<expression> (COMMA <expression>)*
	<case-else-clause>	KEYWORD(selain_itu) <statement-list>
Ekspresi	<expression>	<simple-expression> (RELATIONAL_OPERATOR <simple-expression>)*
	<simple-expression>	<term> ((ARITHMETIC_OPERATOR(

		+ -) LOGICAL_OPERATOR(atau) <term>)*
	<term>	<factor> ((ARITHMETIC_OPERATOR(* / bagi mod) LOGICAL_OPERATOR(dan)) <factor>)*
	<factor>	IDENTIFIER / INT_LITERAL / REAL_LITERAL / CHAR_LITERAL / STRING_LITERAL / (LPARENTHESIS + expression + RPARENTHESIS) / LOGICAL_OPERATOR(tida k) + factor / IDENTIFIER + LPARENTHESIS + <actual-parameter-list> +RPARENTHESIS / [ARITHMETIC_OPERATOR(+ -)] <factor>
	Kami melakukan modifikasi dari spesifikasi. Pada spesifikasi M2, disebutkan bahwa Unary Operator (+/-) menjadi bagian dari <simple-expression>. Namun, hal ini menyebabkan ekspresi seperti a - -b tidak valid (Sedangkan di pascal, ini valid). Sehingga, unary operator kami pindahkan ke <factor>.	
Expression Component	<function-call-node>	IDENTIFIER LPARENTHESIS [<actual-parameter-list>] RPARENTHESIS
	<array-access>	<expression> LBRACKET <expression> RBRACKET
	<parenthesized-expression>	LPARENTHESIS <expression RPARENTHESIS
	<not-factor>	LOGICAL_OPERATOR(tidak) <factor>
	<actual-parameter-list>	<expression> (COMMA <expression>)*

Referensi

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, & tools* (2nd ed.). Pearson/Addison Wesley.

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson/Addison Wesley.

Klabnik, S., & Nichols, C. (2023). *The Rust programming language* (2nd ed.). No Starch Press.